Geavanceerde Computerarchitectuur
Session 5

Christoph Sanders & Wout Corijn

2025-03-10

1 Probleemstelling

In dit labo worden vier verschillende functies geïmplementeerd die operaties uitvoeren op een array: *som*, *product*, *minimum* en *maximum*. De implementatie gebeurt op twee manieren:

- **Synchroon**: Elke array wordt eerst op de CPU gegenereerd, vervolgens naar de GPU overgebracht en daarna verwerkt door een kernel-functie. Dit proces wordt sequentieel uitgevoerd, waarbij elke bewerking pas start nadat de vorige is voltooid.
- **Asynchroon**: Terwijl een array wordt verwerkt op de GPU, wordt de volgende array al voorbereid op de CPU. Dit zorgt ervoor dat de CPU en GPU tegelijkertijd werken, waardoor de totale verwerkingstijd wordt verminderd. De workflow bestaat uit het genereren van een array op de CPU, het migreren naar de GPU en het uitvoeren van de kernel-functie, terwijl parallel de volgende dataset wordt voorbereid.

2 Methode

Het gecontroleerd uitvoeren van asynchrone operaties wordt beheerd met streams. Streams fungeren als tokens die een bepaalde volgorde in operaties afdwingen. Operaties met verschillende streams kunnen in willekeurige volgorde worden uitgevoerd, zowel sequentieel als parallel. Daarentegen worden functies binnen dezelfde stream gedwongen om in de gedefinieerde volgorde sequentieel te worden uitgevoerd.
Om parallelle uitvoering mogelijk te maken, wordt voor elke functie een aparte stream aangemaakt. Tegelijkertijd zorgt het gebruik van dezelfde stream ervoor dat de volgorde van de bijbehorende operaties behouden blijft, terwijl de rest parallel verwerkt kan worden.

3 Resultaat

Voor beide methoden wordt de totale uitvoeringstijd gemeten van de vier operaties. De timing gebeurt pas na synchronisatie, om ervoor te zorgen dat alle GPU-bewerkingen correct zijn afgerond. Voor elke array grootte zijn er 100 metingen uitgevoerd en is hiervan het gemiddelde genomen. De grootte van de array wordt stapsgewijs verdubbeld.

Figuur 1 geeft weer dat de asynchrone uitvoering betere prestaties levert dan de synchrone uitvoering. Terwijl de uitvoeringstijd bij de synchrone methode toeneemt naarmate de array grootte groeit, blijft de asynchrone methode relatief constant. Dit toont de efficiëntie van de parallelle verwerking, waarbij taken asynchroon worden afgehandeld, wat leidt tot een snellere uitvoering.

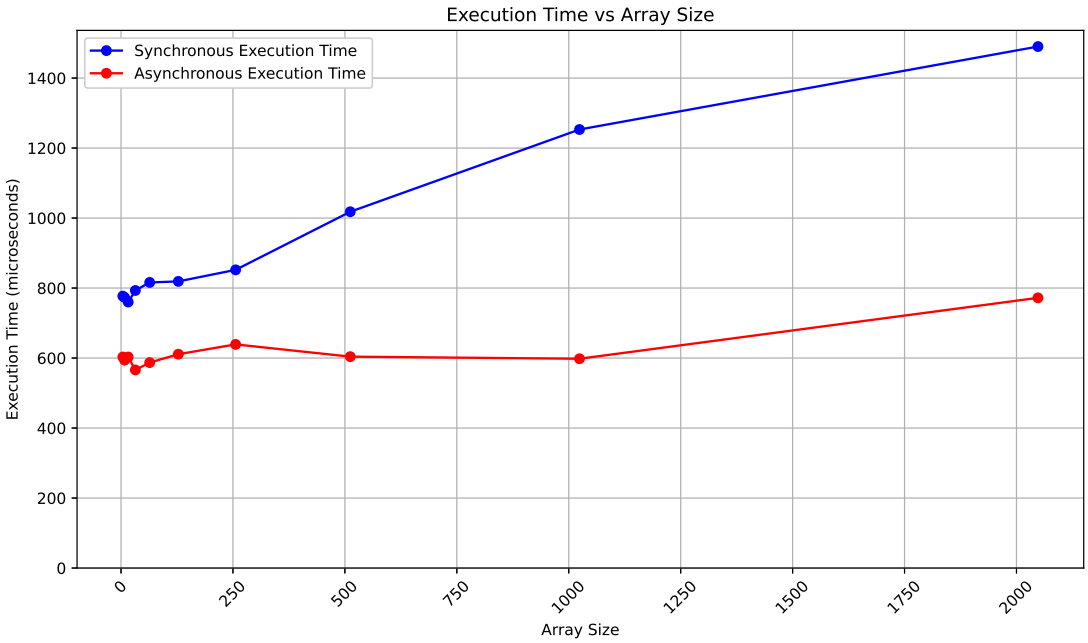


Figure 1: Execution time for varying array size

4 Appendix

4.1 Synchroon

```
__global__ void findMaxValue(int *d_arr, int size, int d_depth, int d_threads) {
    int idx = threadIdx.x;

    for(int i = 0; i < d_depth; i++) {
        int exp = std::pow(2,i);
        if(idx < (d_threads)/exp) {
            int index1 = idx*2*exp;
            int index2 = index1 + exp;

            if(d_arr[index1] < d_arr[index2]) {
                d_arr[index1] = d_arr[index2];
            }
        }
        __syncthreads();
    }
}

__global__ void findMinValue(int *d_arr, int size, int d_depth, int d_threads) {
    int idx = threadIdx.x;
    int offset = blockIdx.x * blockDim.x * 2;

    for(int i = 0; i < d_depth; i++) {
        int exp = std::pow(2,i);
        if(idx < (d_threads)/exp) {
            int index1 = idx*2*exp;
            int index2 = index1 + exp;

            if(d_arr[offset+index1] > d_arr[offset+index2]) {
                d_arr[offset+index1] = d_arr[offset+index2];
            }
        }
        __syncthreads();
    }
}

__global__ void sumArray(int *d_arr, int size, int d_depth, int d_threads) {
    int idx = threadIdx.x;
    int offset = blockIdx.x * blockDim.x * 2;

    for(int i = 0; i < d_depth; i++) {
        int exp = std::pow(2,i);
        if(idx < (d_threads)/exp) {
            int index1 = idx*2*exp;
            int index2 = index1 + exp;

            d_arr[offset+index1] = d_arr[offset+index2] + d_arr[offset+index1];
        }
        __syncthreads();
    }
}

__global__ void prodArray(int *d_arr, int size, int d_depth, int d_threads) {
    int idx = threadIdx.x;
    int offset = blockIdx.x * blockDim.x * 2;

    for(int i = 0; i < d_depth; i++) {
        int exp = std::pow(2,i);
```

```
        if(idx < (d_threads)/exp) {
            int index1 = idx*2*exp;
            int index2 = index1 + exp;

            d_arr[offset+index1] = d_arr[offset+index2] * d_arr[offset+index1];
        }
        __syncthreads();
    }
}



int main() {
    int num_runs = 100;


    // Open CSV file to store the results
    std::ofstream outputFile("sync.csv");
    outputFile << "size, Mean Time in microseconds\n";

    for (int size = 2; size <= 2048; size *= 2) {
        int total_time = 0;

        for (int run = 0; run < num_runs; run++) {
            auto start = std::chrono::high_resolution_clock::now();

            int *d_sum;
            int *d_prod;
            int *d_min;
            int *d_max;

            int* h_arr = (int*)malloc(size * sizeof(int));
            int* h_result = (int*)malloc(size * sizeof(int));
            for (int i = 0; i < size; i++) {
                h_arr[i] = 1;
            }

            int d_depth = std::log2(size);

            cudaMalloc((void**)&d_sum, size * sizeof(int));
            cudaMalloc((void**)&d_prod, size * sizeof(int));
            cudaMalloc((void**)&d_min, size * sizeof(int));
            cudaMalloc((void**)&d_max, size * sizeof(int));

            cudaMemcpy(d_sum, h_arr, size * sizeof(int), cudaMemcpyHostToDevice);
            sumArray<<<1, size/2>>>(d_sum, size, d_depth, size/2);
            cudaMemcpy(h_result, d_sum, size * sizeof(int), cudaMemcpyDeviceToHost
                );

            cudaMemcpy(d_prod, h_arr, size * sizeof(int), cudaMemcpyHostToDevice);
            prodArray<<<1, size/2>>>(d_prod, size, d_depth, size/2);
            cudaMemcpy(h_result, d_prod, size * sizeof(int),
                cudaMemcpyDeviceToHost);

            cudaMemcpy(d_min, h_arr, size * sizeof(int), cudaMemcpyHostToDevice);
            findMinValue<<<1, size/2>>>(d_min, size, d_depth, size/2);
            cudaMemcpy(h_result, d_min, size * sizeof(int), cudaMemcpyDeviceToHost
                );

            cudaMemcpy(d_max, h_arr, size * sizeof(int), cudaMemcpyHostToDevice);
            findMaxValue<<<1, size/2>>>(d_max, size, d_depth, size/2);
```

```
            cudaMemcpy(h_result, d_max, size * sizeof(int), cudaMemcpyDeviceToHost
                );



            cudaFree(d_sum);
            cudaFree(d_prod);
            cudaFree(d_min);
            cudaFree(d_max);

            free(h_arr);
            free(h_result);

            auto end = std::chrono::high_resolution_clock::now();
            auto duration = std::chrono::duration_cast<std::chrono::microseconds>(
                end - start);
            total_time += duration.count();
        }
        int mean_time = total_time / num_runs;
        outputFile << size << "," << mean_time << "\n";
    }

    return 0;
}
```

4.2 Asynchroon

```
__global__ void findMaxValue(int *d_arr, int size, int d_depth, int d_threads) {
    // zelfde code als synchroon
}

__global__ void findMinValue(int *d_arr, int size, int d_depth, int d_threads) {
    // zelfde code als synchroon
}

__global__ void sumArray(int *d_arr, int size, int d_depth, int d_threads) {
    // zelfde code als synchroon
}

__global__ void prodArray(int *d_arr, int size, int d_depth, int d_threads) {
    // zelfde code als synchroon
}

int main() {
    int num_runs = 100;


    // Open CSV file to store the results
    std::ofstream outputFile("async.csv");
    outputFile << "size, Mean Time in microseconds\n";

    for (int size = 2; size <= 2048; size *= 2) {
        int total_time = 0;

        for (int run = 0; run < num_runs; run++) {
            auto start = std::chrono::high_resolution_clock::now();

            int* h_arr = (int*)malloc(size * sizeof(int));
            int* h_result = (int*)malloc(size * sizeof(int));

```

```
34              for (int i = 0; i < size; i++) {
35                  h_arr[i] = 1;
36              }
37
38              int *d_sum;
39              int *d_prod;
40              int *d_min;
41              int *d_max;
42
43              int d_depth = std::log2(size);
44
45              cudaMalloc((void**)&d_sum, size * sizeof(int));
46              cudaMalloc((void**)&d_prod, size * sizeof(int));
47              cudaMalloc((void**)&d_min, size * sizeof(int));
48              cudaMalloc((void**)&d_max, size * sizeof(int));
49
50              cudaStream_t streamSum;
51              cudaStreamCreate(&streamSum);
52              cudaStream_t streamProd;
53              cudaStreamCreate(&streamProd);
54              cudaStream_t streamMin;
55              cudaStreamCreate(&streamMin);
56              cudaStream_t streamMax;
57              cudaStreamCreate(&streamMax);
58
59              cudaMemcpyAsync(d_sum, h_arr, size * sizeof(int),
                    cudaMemcpyHostToDevice, streamSum);
60              sumArray<<<1, size/2, 0, streamSum>>>(d_sum, size, d_depth, size/2);
61              cudaMemcpyAsync(d_prod, h_arr, size * sizeof(int),
                    cudaMemcpyHostToDevice, streamProd);
62              prodArray<<<1, size/2, 0, streamProd>>>(d_prod, size, d_depth, size/2)
                    ;
63              cudaMemcpyAsync(d_min, h_arr, size * sizeof(int),
                    cudaMemcpyHostToDevice, streamMin);
64              findMinValue<<<1, size/2, 0, streamMin>>>(d_min, size, d_depth, size
                    /2);
65              cudaMemcpyAsync(d_max, h_arr, size * sizeof(int),
                    cudaMemcpyHostToDevice, streamMax);
66              findMaxValue<<<1, size/2, 0, streamMax>>>(d_max, size, d_depth, size
                    /2);
67
68              // Synchronize de streams
69              cudaStreamSynchronize(streamSum);
70              cudaStreamSynchronize(streamProd);
71              cudaStreamSynchronize(streamMin);
72              cudaStreamSynchronize(streamMax);
73
74              cudaMemcpyAsync(h_result, d_sum, sizeof(int), cudaMemcpyDeviceToHost,
                    streamSum);
75              cudaMemcpyAsync(h_result, d_prod, sizeof(int), cudaMemcpyDeviceToHost,
                     streamProd);
76              cudaMemcpyAsync(h_result, d_min, sizeof(int), cudaMemcpyDeviceToHost,
                    streamMin);
77              cudaMemcpyAsync(h_result, d_max, sizeof(int), cudaMemcpyDeviceToHost,
                    streamMax);
78
79              cudaStreamDestroy(streamSum);
80              cudaStreamDestroy(streamProd);
81              cudaStreamDestroy(streamMin);
82              cudaStreamDestroy(streamMax);
83              cudaFree(d_sum);
84              cudaFree(d_prod);
```

```
            cudaFree(d_min);
            cudaFree(d_max);
            free(h_arr);
            free(h_result);

            auto end = std::chrono::high_resolution_clock::now();
            auto duration = std::chrono::duration_cast<std::chrono::microseconds>(
                end - start);
            total_time += duration.count();
        }
        int mean_time = total_time / num_runs;
        outputFile << size << "," << mean_time << "\n";
    }

    return 0;
}
```